Arecibo Zero: testing interstellar message designs on minds that start from nothing

Randomly initialised neural networks stand in for the receiver of a first-contact message. They learn arithmetic from the transmission alone, are examined on sums the message never contained, and are then taken apart to read the message back.

PS12007 · technical report · October 2026 · github.com/PS12007/arecibo-zero · interactive demo

Abstract. We test interstellar message designs empirically. A sender writes arithmetic (sums, products, comparisons, counting) under a fixed bit budget, choosing the number system and digit order. The receiver is a randomly initialised transformer or LSTM trained only on that message to predict the next symbol, and it is examined on sums whose operand pairs never appear in the message. Across 434 receivers trained on one laptop GPU: (1) writing the least-significant digit first beats the human order in every base, cutting the bits needed for 50% held-out accuracy by 1.4–4×. Base 4 needs the fewest (about 16k bits), and binary written as in the Arecibo message never reaches 50% within 131k bits. (2) The best numerals depend on the receiver: LSTMs answer 51–83% of unseen sums (three seeds) after hearing just 41 sums in unary, where every positional system leaves them at chance. (3) At a fixed budget, accuracy on sums is predicted by the number of sums alone. Counting gives no head start, and products, which share the sums' form, cost 8–11 points. (4) Delivered as raw bits without symbol boundaries, the same content teaches nothing (about 1% against 91%): framing is the hardest part of the message. (5) Reversing roles, we recover the complete key of 5 of 12 symbol-shuffled messages (base, digit values, digit order, +, × and the comparison) by questioning a trained receiver. A genetic search over designs independently converges on base 4, least-significant digit first for transformers, and on unary for LSTMs. (6) Sent to five kinds of fresh receivers, plain unary sums, which neither search proposed, are the only design that every kind understood at 2^{14} bits (28–56%, against at most 14% on LSTMs for any other design).

1 · Introduction

In 1974 the Arecibo radio telescope sent 1,679 bits towards the globular cluster M13. The message opens with the numbers one to ten written in binary, on the assumption that counting is the first thing any technological species would recognise [1]. Later designs went further: Freudenthal's *Lincos* builds a whole language up from arithmetic statements [2], and the Cosmic Call transmissions of 1999 and 2003 begin with pages of numerals, sums and equalities [3]. Every such message rests on guesses about the receiver: that it will find the symbols, spot the base and digit order, and work out what the operator signs mean. None of these guesses has been tested, because there has never been a receiver to test them on.

This report uses the nearest thing available: a mind that starts with no knowledge at all. A small neural network, randomly initialised, is trained on one transmission and nothing else, purely to

predict the next symbol. It is never told what the symbols are, where statements begin, or that the message is about numbers. Then it is examined on **sums that the message never contained**: a quarter of all operand pairs below 64 are withheld from every design, in both orders. A receiver can answer them only by having extracted the rule of addition from the examples it heard. That turns message design into an experiment: hold the bit budget and the mathematics fixed, change how the message is written, and count how many receivers understand it.

Neural networks are not aliens, and §9 discusses what this proxy can and cannot say. They are, though, the only general learners we can build that have no prior notion of number, and they are a useful adversary: a design that only works if the receiver already thinks like us will fail on them. We ask five questions:

1. **Numerals.** Which way of writing numbers (unary tallies; bases 2, 3, 4, 6, 10, 16; most- or least-significant digit first) is understood with the fewest bits?
2. **Curriculum.** At a fixed budget, does spending bits on counting, comparisons or products help a receiver learn addition, or just dilute it?
3. **Channel.** If the receiver gets only raw, noisy bits, with no symbol boundaries, is repetition coding worth its cost?
4. **Decipherment.** Can we read the message back *out of* a trained receiver (which symbols are digits, the base, the digit order, and which signs mean plus, times and less-than) without looking at the message's key?
5. **Design search.** If the message is evolved to suit one kind of receiver, does it still work for another?

Everything runs on one 8 GB laptop GPU in about ten hours. No data is downloaded: the messages are generated. One trained receiver runs in the browser on the demo page, and visitors can ask it sums it has never heard.

2 · Setup

2.1 The message

A *design* fixes a radix $b \in \{1, 2, 3, 4, 6, 10, 16\}$ (where 1 means unary tallies), a digit order (most-significant first, as humans write, or least-significant first), and a *mix* of statement kinds. The vocabulary is the b digits plus eight operator symbols: plus, times, equals, a comparison sign, less, greater, a list separator and a statement terminator. There are four kinds of statement:

- **sum** `a + b = c ;` with $a, b < 64$, drawn only from the 75% of unordered pairs that are not held out;
- **product** `a × b = c ;` with $a, b < 16$;
- **comparison** `a ? b = R ;` where R is one of the less, greater or equals symbols;
- **counting** `n , n+1 , … , n+5 ;` with $n < 122$.

Statements are sampled independently from the mix until the bit budget is used up. A symbol costs $\log_2|V|$ bits, so a budget buys fewer symbols when the alphabet is larger, and every design pays the same total. Unary writes n as n copies of one digit and zero as nothing at all, so `0 + 3 =`

`3` becomes `+ 111 = 111`. A stretch of a base-10, LSB-first message with every statement kind looks like this (shown in human symbols, digits least-significant first, so "13 + 63 = 76" means 31 + 36 = 67; the receiver sees only opaque token ids):

```
13 + 63 = 76 ; 82, 92, 03, 13, 23, 33 ; 52 + 0 = 52 ; 31 + 71 = 03 ; 71 + 43 = 15 ;
5 + 55 = 06 ; 84 ? 16 = < ; 94 + 36 = 211 ; 54 + 33 = 87 ; 51 × 4 = 06 ; 64 + 94 =
59 ; 25 + 85 …
```

2.2 Receivers

The main receiver is a 2-layer, 4-head transformer with width 128 (pre-LayerNorm, GELU MLP, learned positions; 0.42M parameters) with a 128-symbol context, or 256 for unary, whose numerals are long. The second is a 2-layer LSTM with 256 units. The design search also uses a narrower transformer (width 64), a 4-layer transformer and a 1-layer, 128-unit LSTM. Every receiver is trained from scratch on random windows of its one message for 8,000 steps (batch 64, AdamW, learning rate 10^{-3} with warm-up and cosine decay, weight decay 0.1), long enough to see each symbol of the largest messages about 30 times. Long training matters. Like the small algorithmic tasks where "grokking" was first seen [4], a base-2 receiver can sit at chance on held-out sums for thousands of steps and then generalise: one pilot went from 3% at 2,000 steps to 93% at 8,000. To make the sweeps affordable, up to ten transformers with different messages and seeds are trained at once as one batched model with stacked weights and per-member gradient clipping. This is mathematically the same as training them separately, and it gave 1.3–2× more throughput on this GPU.

2.3 The exam

To ask a question, we show the receiver a real stretch of its message ending at a statement terminator, then the question in its own symbols, for example `a + b =`, and let it continue greedily. The answer is correct only if every digit is right and the next symbol is not a digit. The main metric, **held-out accuracy**, uses up to 400 of the 504 held-out unordered pairs, each in a random order. A receiver that has memorised the message scores zero. We also report accuracy on pairs the message did contain, on all 256 products below 16 × 16, and on comparisons. Chance is far below 1% for sums. Nothing about the exam is shown to the receiver during training.

3 · How should numbers be written?

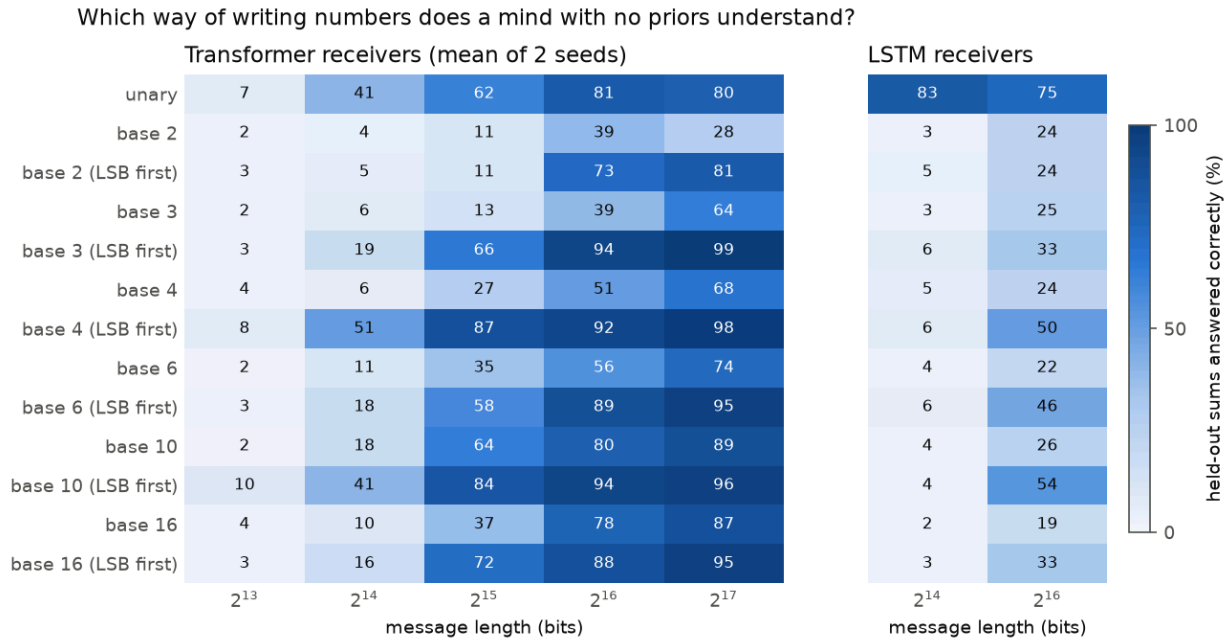


Figure 1. Held-out sum accuracy (%) by numeral system and message length, for transformer receivers (left, mean of two seeds) and LSTM receivers (right, one seed). Every message contains only sums. Each cell is a separately trained receiver, or two.

Figure 1 and Table 1 show the main sweep: 13 numeral systems × 5 message lengths × 2 seeds for transformers (130 receivers), and 2 lengths × 1 seed for LSTMs (26 receivers). Every message contains only sums. Three results stand out.

Least-significant digit first wins in every base. At 2^{15} bits a base-4 message written LSB first gets 87% of held-out sums right and the MSB-first version 27%. The same gap is 66% against 13% for base 3, 72% against 37% for base 16, and 84% against 64% for base 10. In bits needed to reach 50% held-out accuracy (Table 1), MSB-first costs 1.4× (base 10) to 4× (base 4) more. Over all 60 transformer cells where both orders were trained with the same base, length and seed, LSB first is ahead in 47 of the 49 that are not tied at the floor (two-sided sign test $p = 4{\times}10^{-12}$). The mean gain is 21 points (bootstrap 95% interval 16–27). The LSTM cells agree: 7 of 7 untied pairs, mean gain 9 points (`experiments/stats.py`). The reason is plausibly the receiver's own constraint, not the arithmetic: it writes the answer one symbol at a time, left to right. Written LSB first, each digit of the sum depends only on digits already seen and one carry. Written MSB first, the first digit of the answer depends on a carry chain that has not been "computed" yet. This matches what is known for transformers trained on addition [8, 9], but here no one told the receiver what the task was. A human sender would write numbers the way humans do. For this kind of receiver that choice roughly doubles or triples the cost of being understood.

Binary, the Arecibo choice, is the worst positional system. MSB-first binary never reaches 50% within 2^{17} bits (39% and 28% at the two longest lengths, with a large seed spread). LSB-first binary needs about 50k bits, three times as many as LSB-first base 4. Binary numerals are long, so

each sum has more digits and more carries, and the context holds fewer examples. Small bases are not bad in themselves, though: base 4 is the best system overall, ahead of base 10 (16k against 19k bits). The optimum is in the middle: few enough symbols that each digit's role is learned quickly, and short enough numerals that carries are rare.

The best numeral depends on the mind. Unary tallies are mid-table for transformers, with a 22k-bit threshold and a plateau near 80%. For LSTMs they are in a class of their own. With 2^{14} bits, an LSTM given unary sums answers 83% of held-out questions (one seed; two fresh seeds in §7 give 61% and 51%), while every positional system is at 7% or below. That message holds just **41 sums**, covering about 3% of the allowed pairs. Unary addition is concatenation, and a recurrent network can do it with one counter that goes up during the operands and down during the answer. The same counter solves no positional system. At 2^{16} bits the LSTM's positional results are still at most 54% (base 10, LSB first, again ahead of MSB at 26%). A sender who designed for transformers would pick base 4, LSB first. One who designed for LSTMs would pick tally marks, the oldest numerals humans have. Neither design is best for both, and §7 returns to this.

Table 1. Transformer receivers: message length (bits) at which held-out accuracy first reaches 50% (log-interpolated between the lengths tried), and accuracy at 2^{15} and 2^{17} bits. Mean of two seeds.

numerals	bits to 50%	2^{15} bits	2^{17} bits	numerals	bits to 50%	2^{15} bits	2^{17} bits
base 4, LSB first	**16k**	**87%**	98%	base 10	27k	64%	89%
base 10, LSB first	19k	84%	96%	base 16	41k	37%	87%
unary	22k	62%	80%	base 6	54k	35%	74%
base 16, LSB first	25k	72%	95%	base 4	64k	27%	68%
base 3, LSB first	26k	66%	**99%**	base 3	89k	13%	64%
base 6, LSB first	28k	58%	95%	base 2	>131k	11%	28%
base 2, LSB first	50k	11%	81%				

Accuracy on sums that *were* in the message is close to held-out accuracy once a receiver understands (98% against 96% for base 10 LSB at 2^{17} bits), so these receivers are computing, not remembering. Below the threshold both are low: with too few bits the receiver has not even memorised the examples it heard.

4 · What else should the message say?

Every real message design spends bits on more than one idea. The Arecibo message counts from one to ten before anything else, and Lincos introduces comparisons and products alongside sums. Here the numeral system is fixed (base 10, LSB first) and part of the budget goes to counting runs, comparisons or products (30% of statements each, or all three together). Then we ask whether addition gets easier or harder to learn. There are three lengths and three seeds, with 45 transformers and 5 LSTMs.

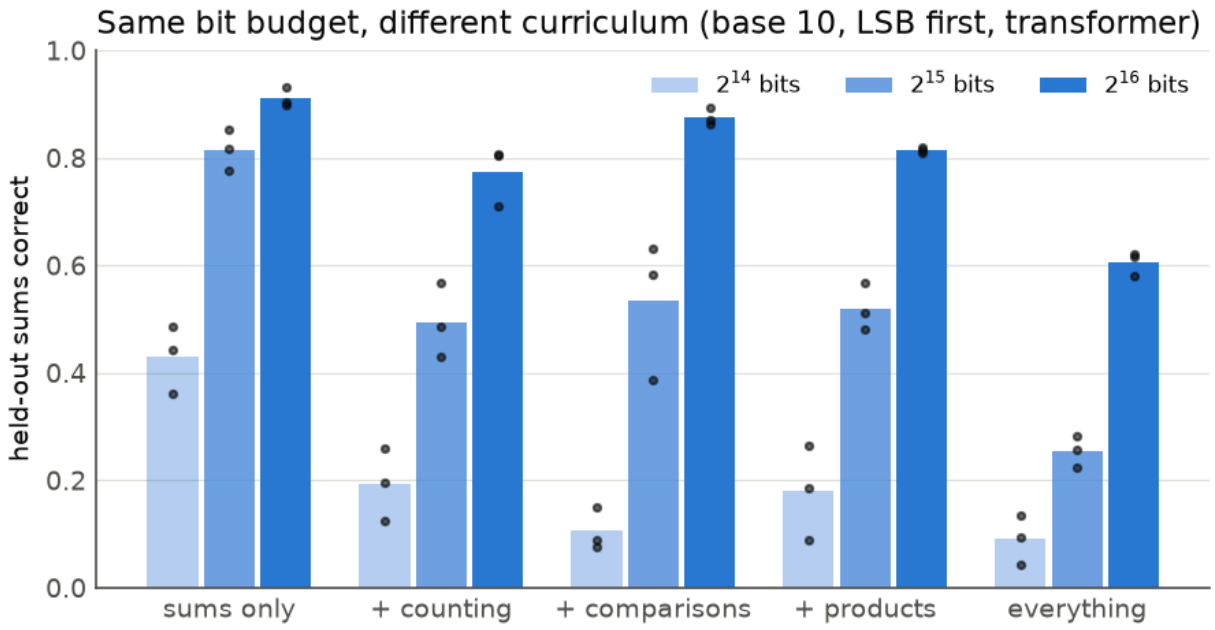


Figure 2. Held-out sum accuracy when part of the same bit budget is spent on other mathematics. Bars are means of three transformer receivers (dots). "Everything" is 40% sums and 20% each of the rest.

At an equal budget, sums alone are always best (Figure 2). The question is whether the other content does anything *beyond* taking bits away from sums. To find out, we predict each receiver's accuracy from the number of sums its message contains alone, by interpolating the sums-only curve, and compare. Table 2 shows the gaps.

Table 2. Held-out sum accuracy minus the accuracy a sums-only message with the same number of sums would reach (interpolated from the 13 sums-only base-10 LSB receivers). Values near zero mean the other content is neutral apart from the bits it takes.

extra content	2^{14} bits	2^{15} bits	2^{16} bits	skill learned at 2^{16}
counting	+0.00	+0.04	−0.08	(counting not examined)
comparisons	−0.18	−0.10	−0.02	89% comparisons
products	−0.11	−0.11	−0.08	83% products
all three	+0.06	−0.02	−0.01	62% products, 84% comparisons

No content helps. Counting, the traditional opening of interstellar messages, gives no head start on addition: a receiver learns sums as if the counting runs were not there. Counting is also expensive. Six consecutive numerals take far more bits than a sum, so 30% of statements is about half the budget. Products hurt a little at every length, by 8–11 points. They share the sum's form, `a ∘ b = c`, and differ only in the operator symbol, so early on the receiver confuses the two. Comparisons cost something only while the receiver is still learning. The receivers do learn the other operations: 89% of comparisons and 83% of products below 16 × 16 at 2^{16} bits. Bits spent on a concept teach that concept, but they do not transfer to its neighbours. A sender with a fixed budget should put the most important idea in alone first. Each extra idea costs what its bits cost,

and a lookalike idea costs a little more. LSTMs are worse across the board at 2^{15} bits (18% with sums only), and every addition lowers them further.

5 · Raw bits through a noisy channel

So far the receiver has been handed clean symbols. A radio receiver gets a stream of bits instead, with no marks between symbols and some bits flipped. Here every symbol of a base-10, LSB-first, sums-only design gets a random 5-bit codeword. The receiver sees only the concatenated bits and is never told the width. Each bit is flipped with probability 0, 1, 3 or 10%. The sender may also send every bit three times, which cuts the number of sums to a third, since repeats count against the same 2^{16}-bit budget. Receivers are the same transformers with a 512-bit context, which covers about 11 statements. Questions are asked in clean bits, and answers are decoded with the sender's codebook.

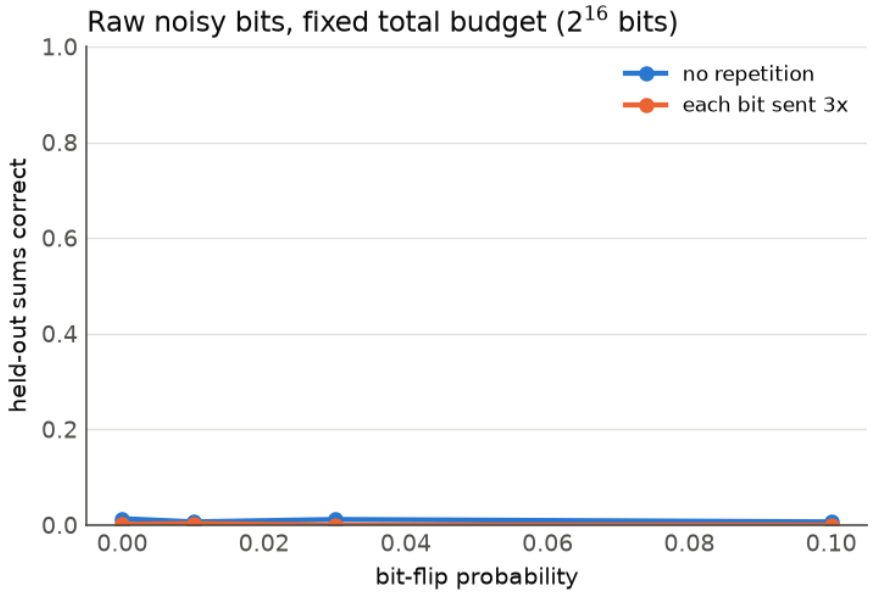


Figure 3. Held-out sum accuracy for receivers trained on raw bits (two seeds per point). Every condition is at the floor.

No receiver learned to add from raw bits, not even with no noise and no repetition (Figure 3). They answer 0–2% of held-out sums and about 1% of sums that *were* in the message. The same content as symbols gives 91%. The receivers did learn something: training loss fell to 0.35 nats per bit, about 1.75 nats per symbol, which captures the framing and the frequent patterns. But it is well above what the sums-only symbol receivers reach, so the arithmetic was never found. With every condition at the floor, the comparison between noise levels and repetition coding cannot be made. A follow-up with four times the training (32,000 steps, clean channel, two seeds) shows what more training buys. Training loss fell tenfold, to 0.033–0.036 nats per bit: the receivers came close to memorising the bit stream. Yet they answer only 4–5% of held-out sums and 11–12% of sums they heard. More training made them better at reciting the message, not at understanding it.

For message design this is the sharpest result in the report. The receivers that learn arithmetic from 2^{14} symbol-level bits learn nothing from 2^{16} raw bits. Finding the symbols is harder than the mathematics. The Arecibo message handles framing by its length, 1,679 = 23 × 73, a product of two primes, which suggests a rectangle. Our results suggest that framing deserves more of the budget than any other design choice.

6 · Reading the message back out of the receiver

The previous sections take the sender's view. This one takes the view of someone who intercepts a message, or of a sender who wants to check that a design could be read back. The question: can we recover the design from a receiver that has understood it, without ever being told the key? For each of six designs (bases 3, 4, 6, 10, 16 with different digit orders) and two seeds, we generate a 2^{17}-bit message containing every statement kind (50% sums, 20% counting, 15% comparisons, 15% products) and **shuffle the symbol labels at random**. A transformer is trained on the shuffled message. The analyst then sees only the shuffled message and the receiver, and proceeds in four steps:

1. **Roles.** Run the receiver over the message and average its predicted next-symbol distribution after each symbol. Digits can follow themselves ("11"); operators almost never can. Symbols whose expected self-repeat probability is at least a tenth of the maximum become seeds. Any symbol whose predicted-continuation profile correlates with the seeds' mean profile ($r > 0.5$) joins them. The result is the set of digits, and so the base.
2. **Ask it to count.** The operator that most often sits between consecutive numerals that end in the same operator is the list separator. At eight places where the message counts, we splice in "*x*, " for each digit *x* and score every one- and two-digit continuation followed by the separator by the receiver's likelihood. The best one is the receiver's answer to "what comes after *x*?"
3. **Values and order.** Following each digit to the first symbol of its successor gives a chain. We try every digit as zero and both digit orders, and keep the assignment under which the most answers are correct successors (two-digit answers such as "9 → 10" or "9 → 01" reveal the digit order). When no chain covers the digits, the order falls back to a weaker cue from the receiver's hidden states (§8), which is how some failed cases still get the order right.
4. **Operators.** With numerals now readable, each pair of operator symbols (*op*, *eq*) is scored by how many statements `a op b eq c` are true under +, and under ×. The comparison operator is the remaining symbol whose single-symbol results depend only on the sign of $a - b$, consistently in more than 95% of cases.

Table 3. Blind decipherment from 12 receivers. "Digits" is digits found / true base. "Counting" means all digit values were recovered exactly. Operators are counted only if the recovered + is true of 100% of decoded statements. The last column is the same receiver's held-out sum accuracy.

design	seed	digits	separator	counting	order	+ and ×	< = >	held-out sums
base 3, MSB first	0	3/3	✓	✗	✓	✗	✗	27%
base 3, MSB first	1	3/3	✓	✗	✓	✗	✗	30%
base 4, LSB first	0	4/4	✓	✓	✓	✓	✓	89%
base 4, LSB first	1	4/4	✓	✓	✓	✓	✓	94%
base 6, MSB first	0	6/6	✓	✗	✓	✗	✗	44%
base 6, MSB first	1	6/6	✓	✓	✓	✓	✓	56%
base 10, LSB first	0	10/10	✓	✓	✓	✓	✓	85%
base 10, LSB first	1	10/10	✓	✗	✗	✗	✗	83%
base 10, MSB first	0	10/10	✓	✓	✓	✓	✓	77%
base 10, MSB first	1	10/10	✓	✗	✓	✗	✗	68%
base 16, LSB first	0	7/16	✓	✗	✗	✗	✗	81%
base 16, LSB first	1	7/16	✓	✗	✗	✗	✗	81%

With the shuffled labels, five of the twelve messages were recovered completely: base, digit values, digit order, the plus and times signs, the comparison sign and its three outcomes. In those five, the recovered addition operator makes 100% of the decoded sum statements true. That is a check an interceptor can run without the key. The probe never misidentifies a separator, and it finds the digits exactly in every base up to 10.

Failures come from two places. **Receivers that have not understood** cannot be read: the two base-3 MSB receivers get only 27–30% of held-out sums, and the counting probe gets no consistent chain of answers from them. The same happens for the weaker of the base-6 MSB receivers (44%) and one base-10 MSB receiver (68%). Every receiver that was read completely scores at least 57% on held-out sums. **The probe itself** is the other source. In base 16 with numbers below 128, only the digits 0–7 ever begin a two-digit numeral, and the role step found exactly those seven, missing the other nine, although these receivers answer 81% of held-out sums. A base-10 LSB receiver with 83% held-out accuracy gave one wrong successor too many. Better probes would read more of these minds. Understanding is necessary for decipherment but, with this probe, not sufficient. For a sender, the corollary is welcome: a design that receivers learn well, such as base 4 or base 10 written LSB first, can also be read back reliably.

7 · Evolving the message, and whether it travels

Finally we let the message design itself. A genome is a base (2–16), a digit order and weights for the four statement kinds. A population of 8 designs is scored by the mean held-out sum accuracy of two transformer receivers at 2^{14} bits. The best 3 survive each generation, and 5 mutants replace the rest. The starting population contains two human designs, decimal sums as humans write them and Arecibo-style binary with counting, plus six random ones.

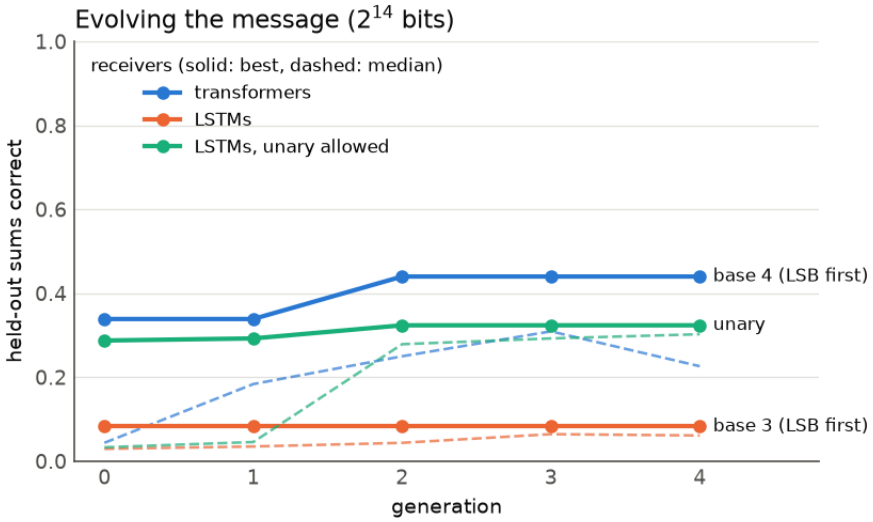


Figure 4. Design search: best and median held-out sum accuracy of the population in each generation, at 2^{14} bits.

The search rediscovered the main result on its own. After four generations every one of the top six designs is base 4, written least-significant digit first. The best scores 44%, against 18% for the human decimal design and 5% for Arecibo-style binary. The champion keeps some counting (23% of statements). But it does not beat the simple sums-only base-4 LSB message from §3, which scored 51% at the same budget, so with two receivers per evaluation the extra counting is within noise rather than a discovery. Designs that add products, or that write numbers MSB first, are eliminated early.

The LSTM search found nothing to climb. Run against two LSTM receivers in the same search space, which excludes unary, the best design (base 3, LSB first, sums only) scored 8% in generation 0 and never improved. At 2^{14} bits every positional system leaves LSTMs at chance (§3), so the fitness landscape is flat.

Allowed to use tally marks, the LSTM search found them at once. A random unary design in generation 0 scored 29%, against at most 5% for the rest, and by generation 2 all eight designs were unary. This is the outcome §3 predicts. But the search did not find the right *content*. Its champion keeps comparisons and counting (32% in the search). Tally-mark statements are long, so those extra statements cut the message to 13–22 sums, against 41 for pure sums. Two receivers per evaluation also make the scores noisy: the same design scored 10% and 54% on two seeds. Selecting on noisy scores favours lucky designs, so every champion scores lower on fresh receivers.

Table 4. Transfer: each design sent to fresh receivers (seeds 5 and 6, never used in a search) of five kinds, at 2^{14} bits. Held-out sum accuracy, mean of two seeds.

message	design	small TF	TF	deep TF	small LSTM	LSTM	mean
unary sums (§3)	unary · add1	28%	40%	39%	54%	56%	**43%**
champion, tuned on transformers	b4-lsb · add0.76+count0.23	14%	41%	16%	3%	5%	16%
champion, LSTMs with unary allowed	unary · add0.89+cmp0.46+count0.18	6%	5%	7%	13%	14%	9%
human: decimal sums	b10 · add1	16%	22%	9%	3%	3%	11%
champion, tuned on LSTMs	b3-lsb · add0.56	9%	15%	12%	3%	5%	9%
Arecibo-style: binary + counting	b2 · add1+count0.3	3%	3%	5%	2%	4%	3%

The most portable message is the oldest numeral. Plain unary sums, which neither search proposed, are understood by every kind of receiver (Table 4): 54–56% by LSTMs and 28–40% by transformers, 43% on average. The transformer champion matches it only on the kind of receiver it was tuned on (41% against 40%) and falls to 14–16% on smaller and deeper transformers and to 3–5% on LSTMs. Human decimal sums average 10%, and the Arecibo-style binary message 3%, at the floor for every receiver. Tuning a message to one mind over-fits it to that mind. Two qualifications apply. The comparison is at the smallest budget only: with 2^{15} bits or more, base 4 written LSB first is far ahead for transformers (87% against 62%, §3). And the fresh LSTM seeds give 51–61% for unary, not the 83% of the single seed in §3, so that number was a lucky draw.

8 · Things that did not work, and caveats

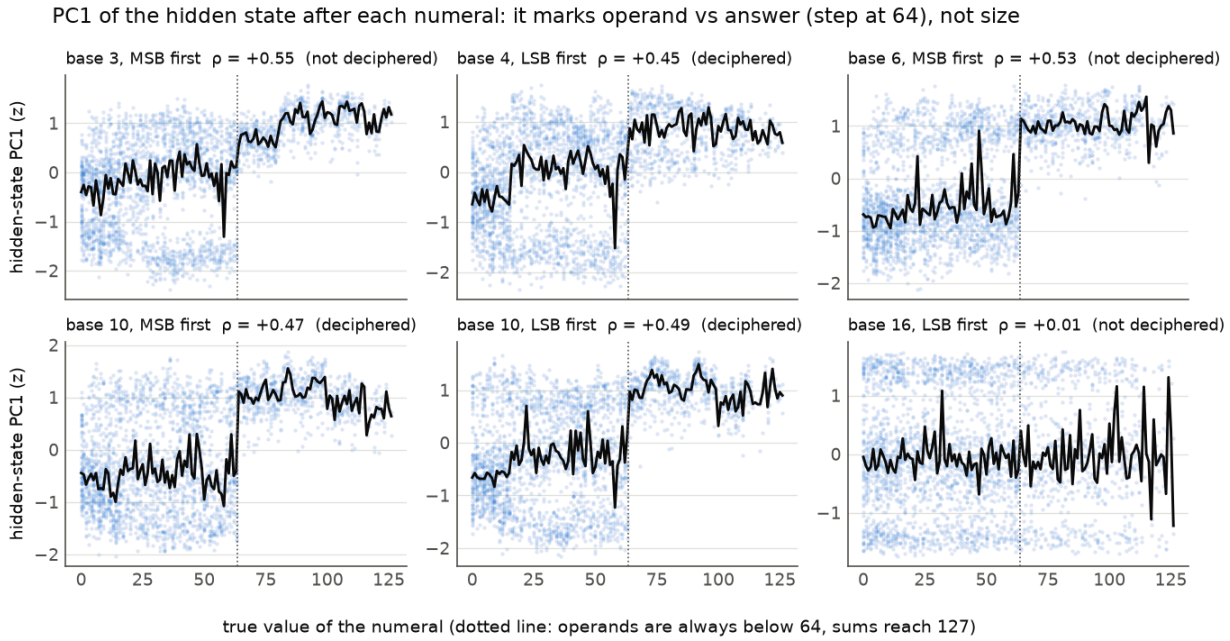


Figure 5. The receiver's "number line": the first principal component of the hidden state after each numeral (3,000 numerals, seed-0 receivers), against the numeral's true value. Black line: median per value. ρ is the rank correlation over all numerals. "Deciphered" refers to Table 3.

- **Reading values from the receiver's geometry did not work reliably.** The first plan was to recover digit values from the first principal component of the hidden state at the end of each numeral, the receiver's "number line". Its rank correlation with the true digit values ranged from −1.0 to 0.64 across the twelve receivers. That is informative about digit order in some cases and useless in others. The counting probe replaced it, and geometry is kept only as a fallback for digit order. Figure 5 shows why. In five of the six receivers PC1 jumps where values cross 64. Operands are always below 64 and sums go up to 127, so the main axis of variation marks *where in the statement* a numeral sits (operand or answer), not how large it is. PC1 correlates with that split at r = 0.47–0.60, and with size inside the operand range at only ρ = 0.01–0.16. The base-16 receiver shows neither. A probe for size would first have to factor out a numeral's role.
- **A better role probe for base 16 found the digits but not a rule.** The receivers have essentially memorised their message: with the full context the median probability of the top prediction is 1.0, and it is still 0.99 for base 10 with only 8 symbols of context, enough to locate a passage in a 28k-symbol message. A memorising receiver is never uncertain, so cues built from its expectations see little. With just 2 symbols of context it is uncertain again, and digits share probability mass where a new operand begins. In both base-16 receivers, all nine digits that the role step missed then share more mass with the seed digits (0.35–0.46) than any operator does (≤ 0.30). But no threshold on that score, absolute, relative to the seeds, or second-order, worked for all six designs without also admitting operators elsewhere, so Table 3 uses the original rule.

- **Simple role heuristics failed.** Separating digits from operators by symbol entropy or frequency did not work. Using self-repetition alone wrongly called "=" a digit, because comparisons produce "= =", and missed rare base-16 digits. The seed-and-correlate rule in §6 fixes the first problem but not the second.
- **Exam resolution.** Each exam asks 400 held-out sums, so a single score has a binomial standard error of at most 2.5 points. Seed-to-seed differences are larger than that: the median gap between the two seeds of a transformer cell is 3 points, and the largest is 44.
- **Seed variance is large near the threshold.** The two LSB-first binary receivers at 2^{16} bits scored 53% and 94%. Delayed generalisation makes the point where a receiver "gets it" sensitive to the seed, so thresholds in Table 1 are rough to within a factor of about 1.5.
- **The batched trainer was cross-checked only near the floor.** At 2^{13} bits single and batched receivers agree on average (mean difference 0.2 points, $r = 0.87$ across ten designs), but there both are near chance. A pilot at 2^{16} bits (base 10 LSB) gave 90–93% single against 94% batched.
- **One evolve stage stalled.** During a period of low system memory the first evolve run made no progress for five hours with the GPU fully busy. The same jobs then ran at the normal speed when restarted. No results were lost. The stall looks like a driver or paging problem, not the code.
- **The exam is not cold, but it does not need to be warm.** Each exam question follows a stretch of real message. To see how much that matters we asked the twelve decipherment receivers (§6) 200 held-out sums each, preceded by 0, 1, 4 or 16 whole statements (`experiments/cold_exam.py`). A bare question is answered much worse: 48% on average against 71% after one statement. But one statement is all it takes, and 16 add nothing (68%). Even a bare question preceded only by the end-of-statement symbol scores 71%, a gain of 14–36 points per receiver over the truly bare question. So the receivers do not learn from examples in the context. They need to know where a statement begins. This is the framing problem of §5 again, in miniature.

9 · Limitations

Neural networks are not aliens. Two architecture families trained by gradient descent are a narrow sample of possible minds, and §3 already shows that the best design changes between them. The conclusions are best read as "this design is robust (or fragile) across the minds we could test", not as predictions about any real receiver. A design that fails here relies on priors that a learner without them cannot recover from this many bits. That is worth knowing, whoever the receiver is.

Arithmetic only. Real messages also carry pictures (the Arecibo raster), physics and chemistry. Numbers come first in every proposal, and they are the easiest to examine with held-out questions, so we start there. The number range is small (sums below 128, products below 256).

Budgets. Even the best design needs about 16,000 bits to reach 50% on held-out sums, and nothing learned anything at 8,192. The 1,679 bits of the Arecibo message would teach these

receivers nothing about arithmetic. A patient receiver that re-reads a short message forever might do better than our fixed 8,000 training steps suggest. We did not test that, and it would change the trade-offs between length and repetition.

Small searches. The design searches (§7) use populations of 8 over 4 generations, two receivers per evaluation and the smallest budget. With two receivers, seed noise is as large as the differences being selected on, so champions regress on fresh receivers. The searches find a good region (base 4 LSB first; unary) but not the best design within it.

10 · Related work

Message design. The Arecibo message [1], Lincos [2], the Cosmic Call messages [3] and many other proposals [5, 6] all teach numbers first, and argue for a particular base and layout. Their arguments are about what a receiver *ought* to find obvious. Minsky argued that any intelligence would converge on arithmetic [7]. As far as we know this is the first work to test candidate designs empirically, against learners that start with no priors, using held-out generalisation as the measure of understanding.

Arithmetic in transformers. Writing the answer least-significant digit first is known to help transformers that are *trained to do* addition [8, 9], as are position schemes that line digits up [10]. Those works train on question-answer pairs with a known format. Here the receiver gets a single unlabelled stream, chosen by a sender under a bit budget, and must discover the format, the base and the operation itself. Grokking [4] and its mechanistic explanation [11] describe the delayed generalisation we also see.

Decipherment and emergent communication. Statistical decipherment recovers lost scripts and substitution ciphers from text [12, 13]. Work on emergent communication studies codes that neural agents *invent* [14]. In this work one side's code is fixed, and the other side is a learner whose internals we then probe to recover it.

11 · Conclusion

Treating message design as an experiment, with untrained minds as receivers and held-out questions as the test of understanding, gives clear and sometimes surprising answers. Write the least-significant digit first: it wins in every base, by up to a factor of four in bits. Use a small but not tiny base: base 4 beats both decimal and the binary of the Arecibo message, which these receivers can barely learn. Do not expect one design to suit every mind: given 2^{15} bits or more, base 4 written LSB first is best for transformers, while tally marks are by far the best for LSTMs. At the smallest budget, plain tally marks were the only design every kind of receiver understood, and a message tuned to one kind of receiver did poorly on the others. Spend bits on the idea you want understood, because counting and other neighbouring ideas do not transfer. Most of all, solve framing first: with symbol boundaries removed, the same receivers learned nothing from four times the bits. The receivers can also be turned around. A network that has understood a message can be questioned until it gives the message's key back, and this worked completely for

five of twelve shuffled designs. The trained receiver on the demo page answers sums that were never transmitted to it. That is, in a small way, what a first contact message is supposed to achieve.

References

1. The Staff at the National Astronomy and Ionosphere Center. The Arecibo message of November, 1974. *Icarus* 26(4):462–466, 1975.
2. H. Freudenthal. *Lincos: Design of a Language for Cosmic Intercourse*. North-Holland, 1960.
3. Y. Dutil and S. Dumas. The Evpatoria messages (Cosmic Call 1999 and 2003). Interstellar message design notes, 1999–2003.
4. A. Power, Y. Burda, H. Edwards, I. Babuschkin, V. Misra. Grokking: generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, 2022.
5. D. A. Vakoch (ed.). *Archaeology, Anthropology, and Interstellar Communication*. NASA SP-2013-4413, 2014.
6. C. Sagan, F. Drake, A. Druyan, T. Ferris, J. Lomberg, L. Salzman Sagan. *Murmurs of Earth: The Voyager Interstellar Record*. Random House, 1978.
7. M. Minsky. Communication with alien intelligence. *Byte* 10(4), 1985.
8. N. Lee, K. Sreenivasan, J. D. Lee, K. Lee, D. Papailiopoulos. Teaching arithmetic to small transformers. ICLR 2024.
9. R. Nogueira, Z. Jiang, J. Lin. Investigating the limitations of transformers with simple arithmetic tasks. arXiv:2102.13019, 2021.
10. S. McLeish et al. Transformers can do arithmetic with the right embeddings. NeurIPS 2024.
11. N. Nanda, L. Chan, T. Lieberum, J. Smith, J. Steinhardt. Progress measures for grokking via mechanistic interpretability. ICLR 2023.
12. B. Snyder, R. Barzilay, K. Knight. A statistical model for lost language decipherment. ACL 2010.
13. S. Ravi and K. Knight. Deciphering foreign language. ACL 2011.
14. A. Lazaridou and M. Baroni. Emergent multi-agent communication in the deep learning era. arXiv:2006.02419, 2020.